

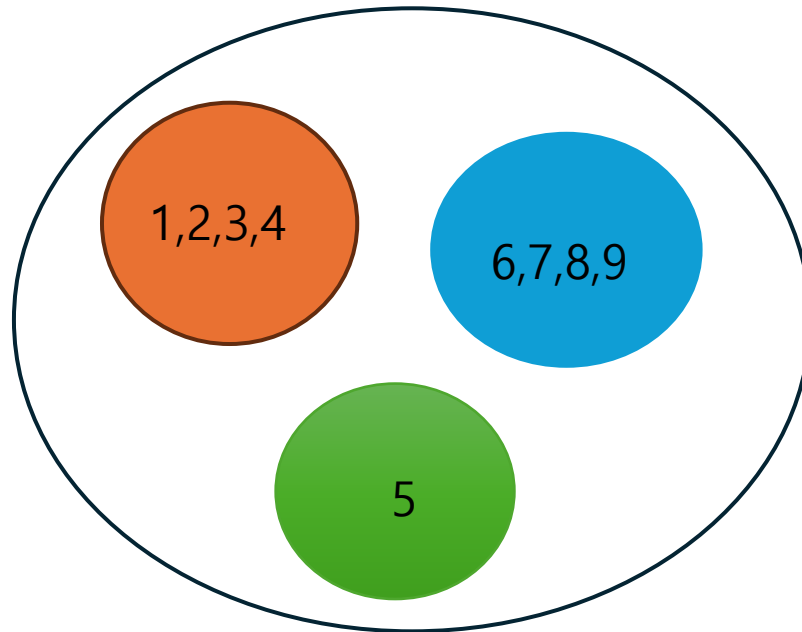
자구&알고 스터디

분리 집합 (Disjoint Set Union, Union Find)

이분 탐색 (Binary Search)

분리 집합

- 서로의 교집합이 공집합인 집합의 모음을 효율적으로 표현하는 자료구조.
- 각 원소가 특정한 집합 한 가지에 있는 상태를 효율적으로 관리 가능
- 가장 대표적인 케이스: 그래프에서 두 원소가 연결되어 있는가?



분리 집합

- 분리 집합 필수 메소드
 - Union
 - Find
 - 두 함수 때문에 분리 집합을 활용해 푸는 문제를 유니온-파인드(Union-Find)문제라고 하기도 함.
- Union 함수
 - 두 집합을 합침
- Find 함수
 - 특정 원소가 어느 집합에 있는지 확인

분리 집합

- 구현 방법:
 - 배열 활용
 - map 활용

```
// map  
// key: 원소  
// value: 원소가 속해 있는 집합  
// 자료형은 필요에 따라 바꾸자
```

```
map<int, int> dsu;
```

```
// 배열  
// MAX는 최대 원소의 개수 만큼  
// 이때 원소는 배열의 인덱스  
// 배열에 저장되는 값은 집합
```

```
int dsu[MAX];
```

분리 집합

Union 함수

```
// x 집합과 y 집합을 합침  
// 통상적으로 값이 더 작은 루트 노드를,  
// 즉 x와 y 중 작은 원소를 부모로 설정  
// 'union' 이라는 키워드가 이미 있음.
```

```
Void _union(int x, int y) {  
    x = find(x);  
    y = find(y);  
    if(x < y) dsu[y] = x;  
    else dsu[x] = y;  
}
```

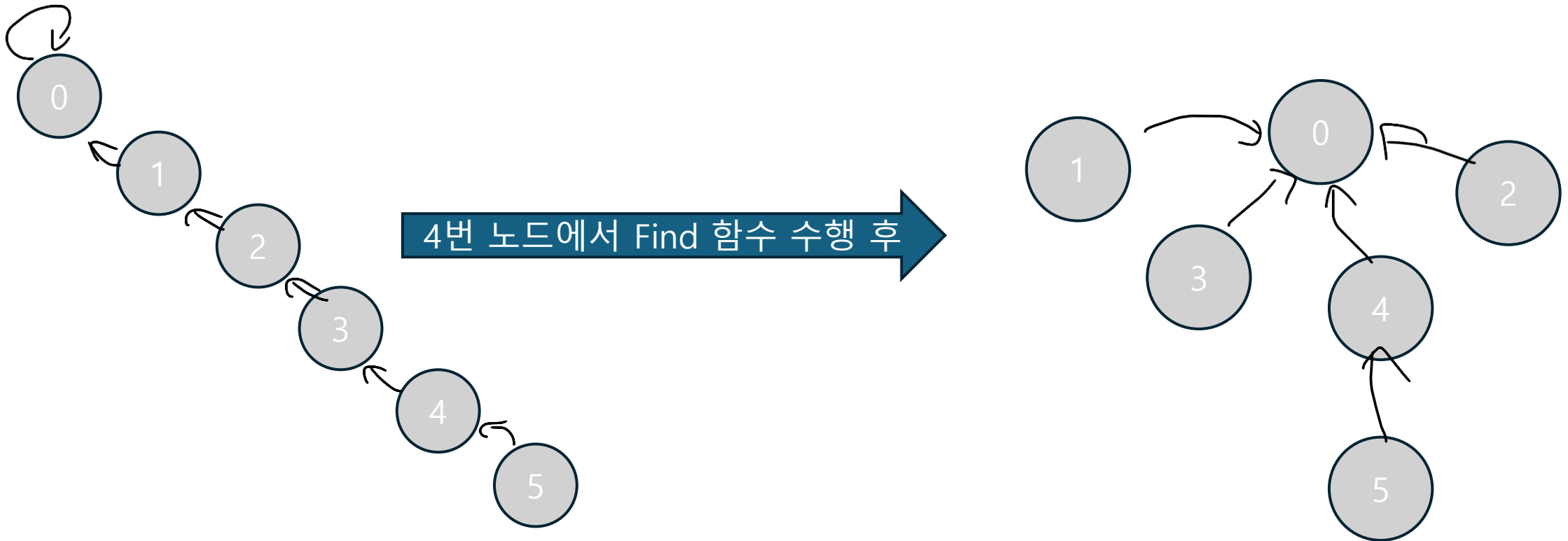
Find 함수

```
// 경로 압축까지 구현된 find 함수  
// 경로 압축을 하게 되면 반복적으로  
// 같은 원소의 집합을 찾을 때 더 효율적이다.
```

```
int _find(int toFind){  
    // 경로 압축!  
    if (dsu[toFind] != toFind)  
        dsu[toFind] = find(dsu[toFind]);  
  
    return dsu[toFind];  
}
```

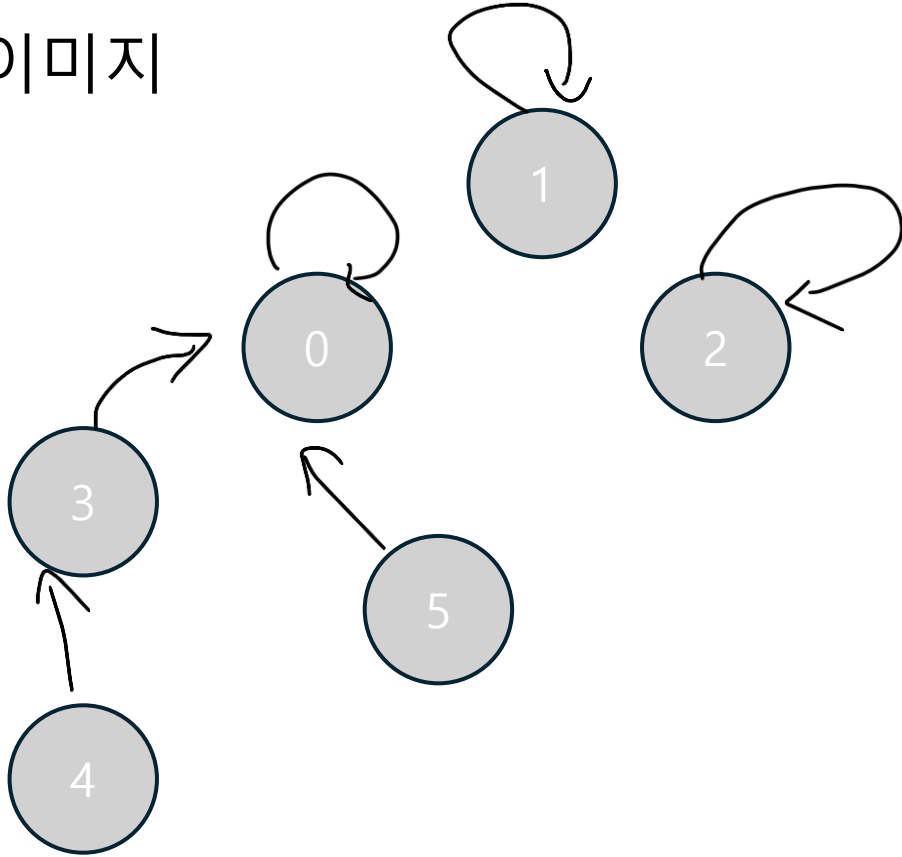
분리 집합

- 경로 압축은 Find 함수 과정에서 실행할 수 있는 분리 집합 최적화 방법.
 - 간단하게 이해하면, find 함수를 호출할 때 노드가 루트 노드가 아닌 경우, 해당 노드를 집합의 루트 노드로 가리키게 업데이트 하는 방법.



분리 집합

이미지

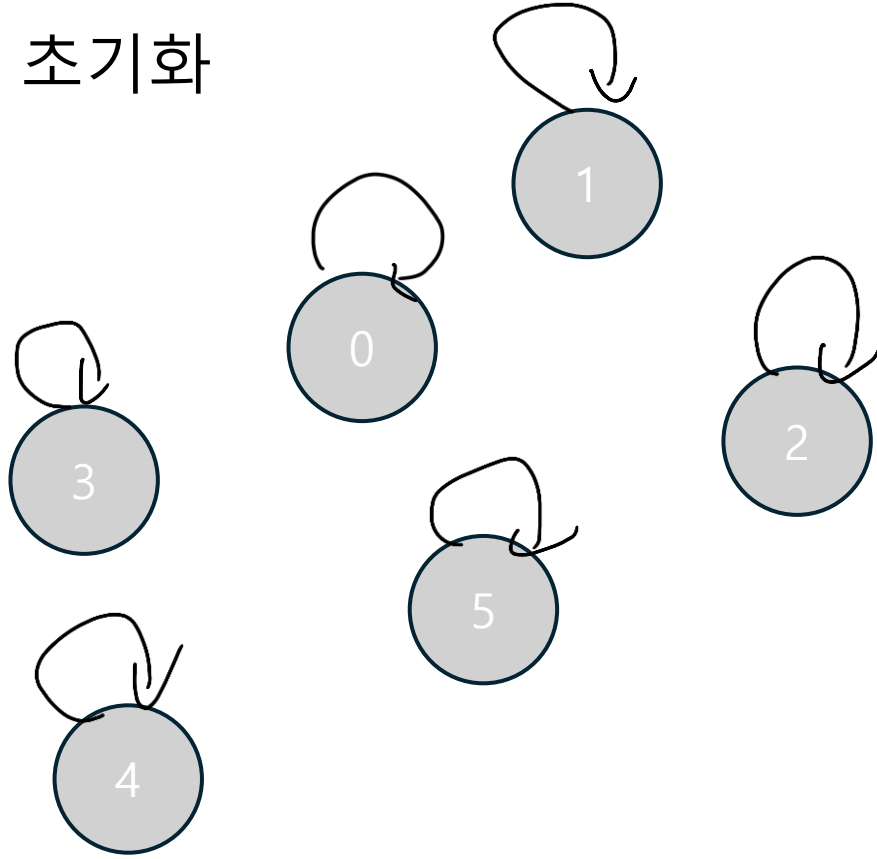


인덱스	0	1	2	3	4	5
소속 집합	0	1	2	0	3	0

- 인덱스와 소속 집합이 다르다.
 - 해당 인덱스는 가리키는 인덱스와 같은 집합에 속해 있다
 - 이런 경우 해당 인덱스가 속해 있는 집합은 원소가 최소 2개
- 인덱스와 소속 집합이 같다
 - 해당 인덱스는 집합의 루트 라고 생각하면 편함
 - 해당 인덱스가 속해 있는 집합은 원소가 1개 또는 여러 개 일 수 있음.

분리 집합

초기화

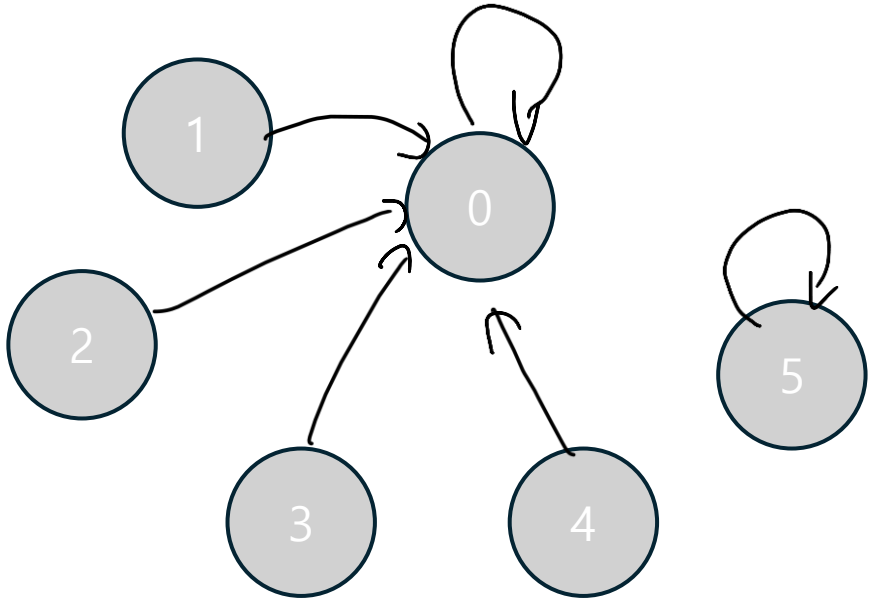


인덱스	0	1	2	3	4	5
소속 집합	0	1	2	3	4	5

- 보통 분리집합을 초기화 할 때에는 각 원소가 각자의 집합의 루트 원소로 선언 시킴
 - 즉 n 개의 원소를 통해 n 개의 집합을 만들어 놓고 시작함
- 각 원소는 처음에 서로 다른 집합에 속해있다고 생각!

분리 집합

- Find 함수 과정 (최적)

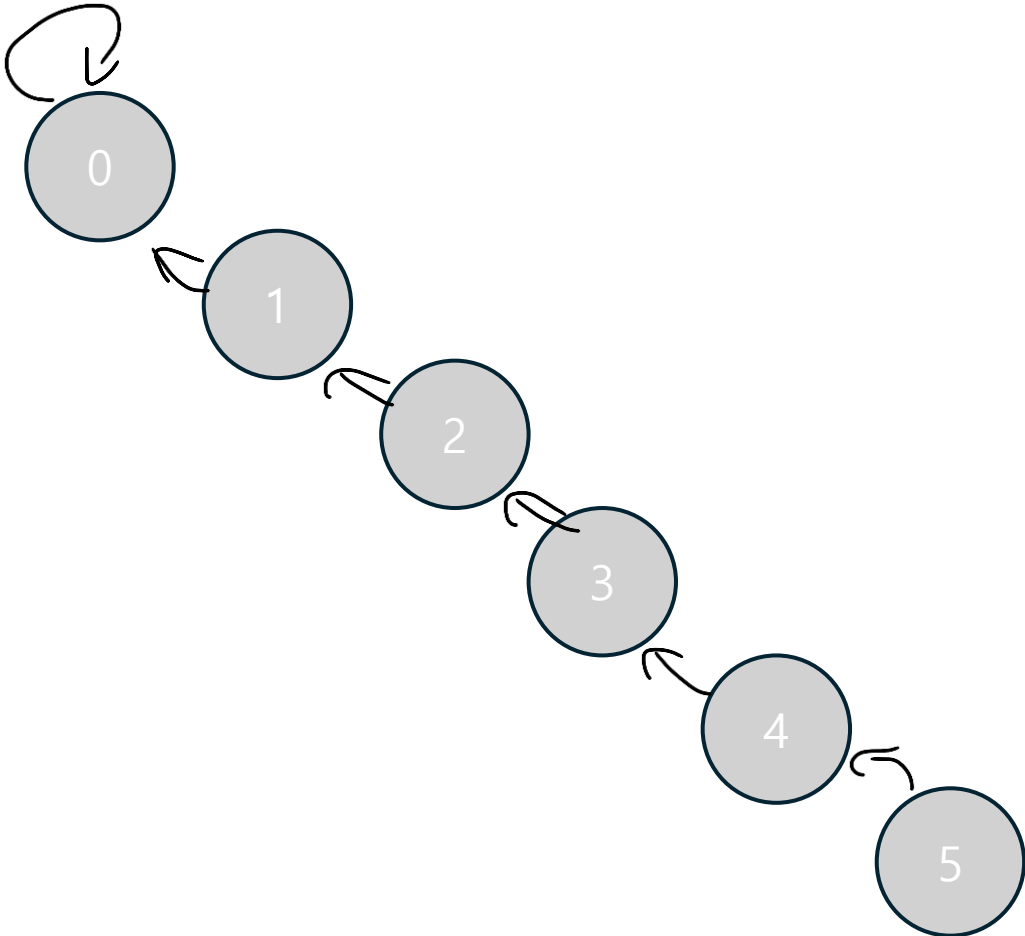


인덱스	0	1	2	3	4	5
소속 집합	0	0	0	0	0	5

어떤 경우의 수라도 재귀를 1번 이하 반복하면 찾을 수 있음

분리 집합

- Find 함수 과정 (최악)

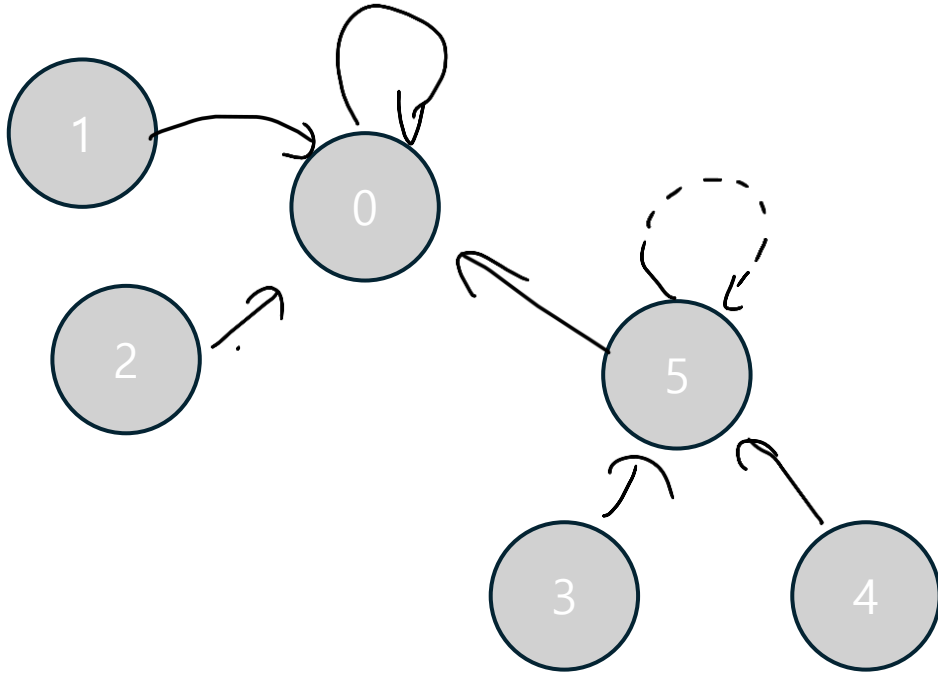


인덱스	0	1	2	3	4	5
소속 집합	0	0	1	2	3	4

인덱스 5의 집합을 찾는 경우, 총 n번 재귀함수를 호출해야 찾을 수 있음.

분리 집합

- Union 함수 과정 (최적)

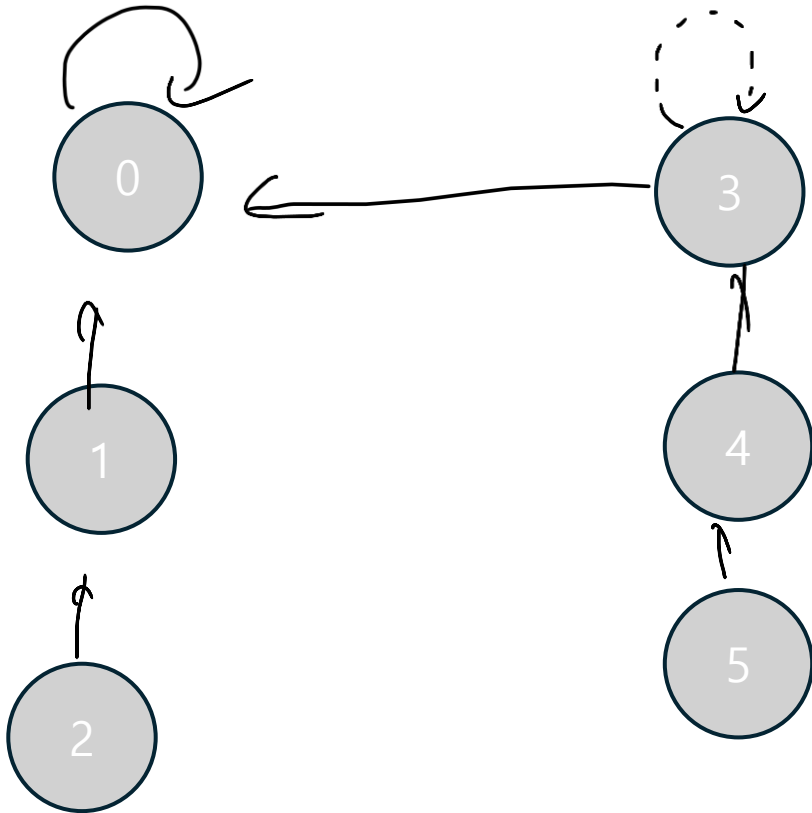


인덱스	0	1	2	3	4	5
소속 집합	0	0	1	5	5	0

- Find 함수가 최적인 상황에서는 Union 함수도 덩달아 효율적으로 변환

분리 집합

- Union 함수 과정 (최악)



인덱스	0	1	2	3	4	5
소속 집합	0	0	1	2	3	4

- Find 함수가 최악인 상황에서, 각 집합이 리프 노드 끼리 Union 하게 된다면 Union 함수의 성능이 많이 떨어짐.

분리 집합

- 예제 문제: <https://www.acmicpc.net/problem/24391>
 - 예시 답안 : <http://boj.kr/f233b6d383b44f5895f635dbb342d4e0>

이분 탐색

- 원하는 값을 찾을 때 까지 탐색 범위를 반으로 줄이면서 효율적으로 탐색하는 알고리즘
- 이분 탐색은 다양한 알고리즘에 활용되고, 심지어 조건만 잘 맞춘다면 다양한 문제에 적용할 수 있다.
 - 정렬된 배열에서 특정 값의 위치를 찾아라 : 정직하게 이분 탐색
 - 특정 범위에서 (또는 구간에서) T가 있을 수 있는가? : 이분 탐색을 활용해서 $arr[i] \leq T < arr[i+1]$ 찾기
 - 배열에서 T 보다 작은(큰) 값들 중 최솟값(최댓값)은? : 이분 탐색으로 $arr[i] \leq T < arr[i+1]$ 찾기
 - 심지어 이분탐색을 활용해 구현되는 알고리즘도 다수 존재: LIS 등
- 저장되어 있는 내용이 매우 큰 경우, 또는 특정 조건을 만족시키는 값을 빠르게 찾고 싶은 경우 이분 탐색을 활용하면 됨.
 - 예를 들어, 매우 큰 배열에서 특정 값 20 을 찾아야 한다면, 처음부터 훑어 보는 것 보다는 한 번 정렬한 뒤 이분탐색해서 찾는 것이 더 빠름.

이분 탐색

코드

`vector<int> arr; // 전역변수. 정렬 되어 있다.`

```
int search(int start, int end, int key) {  
    while (start <= end) {  
        int middle = start + (end - start) / 2;  
        if (arr[middle] < key) {  
            start = middle + 1;  
        } else if (key < arr[middle]) {  
            end = middle - 1;  
        } else {  
            return middle;  
        }  
    }  
    return -1; // 찾기 실패  
}
```

그림 (3을 찾는 경우)

1	2	3	4	5	6	7
---	---	---	---	---	---	---

- 처음 시작할 때 $start = 0$, $end = 6$ $middle = 3$ 이다.
- 이때 $3 < arr[middle]$ 이므로 $end = middle - 1$ 로 업데이트

1	2	3	4	5	6	7
---	---	---	---	---	---	---

- $start = 0$, $end = 2$ 이므로 $middle = 1$ 이다.
- 이때 $3 > arr[middle]$ 이므로 $start = middle + 1$ 로 업데이트

1	2	3	4	5	6	7
---	---	---	---	---	---	---

- $start = 2$, $end = 2$ 이므로 $middle = 2$ 이다.
- 이때 $3 == arr[middle]$ 이므로 우리는 원하는 값을 찾았다.

3이 없는 경우

1	2	3.5	4	5	6	7
---	---	-----	---	---	---	---

- $start = 2$, $end = 2$ 이므로 $middle = 2$ 이다.
- 이때 $3 < arr[middle]$ 이므로 $end = middle - 1$ 로 업데이트
- 하지만 그렇게 되면 $start > end$ 가 되어버리므로 실패

이분 탐색

- 정직하게 이분 탐색을 하게 되면, 중복되는 값이 여러 개 있을 때, 결과가 달라질 수 있음
 - $[1, 2, 2, 2, 5]$ 에서 2를 이분탐색으로 찾을 경우 결과가 1, 2, 3중 나올 수 있게 됨.
- Lowerbound 이분 탐색: $key \leq arr[i]$ 를 만족시키는 최소 인덱스 i 찾기
 - $[1, 2, 2, 2, 5]$ 에서 2를 lowerbound 이분탐색으로 찾을 경우 1 반환
- Upperbound 이분 탐색: $key < arr[i]$ 를 만족시키는 최소 인덱스 i 찾기
 - $[1, 2, 2, 2, 5]$ 에서 2를 upperbound 이분탐색으로 찾을 경우 4 반환
 - 이때, 2의 마지막 인덱스(또는 $key > arr[i]$ 인 값)는 $upperbound(2) - 1$ 임을 알 수 있다.

이분 탐색

lowerbound

`vector<int> arr; // 전역변수 정렬 되어 있다.`

```
int lowerbound(int start, int end, int key) {  
    while (start <= end) {  
        int middle = start + (end - start) / 2;  
        if (key <= arr[middle]) {  
            end = middle - 1;  
        } else {  
            start = middle + 1;  
        }  
    }  
    return start;  
}
```

그림 (3을 찾는 경우)

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- 처음 시작할 때 $start = 0$, $end = 6$ $middle = 3$ 이다.
- 이때 $3 \leq arr[middle]$ 이므로 $end = middle - 1$ 로 업데이트

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- $start = 0$, $end = 2$ 이므로 $middle = 1$ 이다.
- 이때 $3 > arr[middle]$ 이므로 $start = middle + 1$ 로 업데이트

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- $start = 2$, $end = 2$ 이므로 $middle = 2$ 이다.
- 이때 $3 \leq arr[middle]$ 이므로 $end = middle - 1$ 로 업데이트.

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- $start = 2$, $end = 1$ 이므로 반복문 탈출
- $start$ 가 $3 \leq arr[i]$ 를 만족하는 i 의 최솟값인 것을 확인 가능

이분 탐색

upperbound

`vector<int> arr; // 전역변수 정렬 되어 있다.`

```
int upperbound(int start, int end, int key) {  
    while (start <= end) {  
        int middle = start + (end - start) / 2;  
        if (arr[middle] <= key) {  
            start = middle + 1;  
        } else {  
            end = middle - 1;  
        }  
    }  
    return start;  
}
```

그림 (3을 찾는 경우)

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- 처음 시작할 때 $start = 0$, $end = 6$ $middle = 3$ 이다.
- 이때 $arr[middle] \leq 3$ 이므로 $start = middle + 1$ 로 업데이트

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- $start = 4$, $end = 6$ 이므로 $middle = 5$ 이다.
- 이때 $arr[middle] > 3$ 이므로 $end = middle - 1$ 로 업데이트

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- $start = 4$, $end = 4$ 이므로 $middle = 4$ 이다.
- 이때 $arr[middle] \leq 3$ 이므로 $start = middle + 1$ 로 업데이트

1	2	3	3	3	6	7
---	---	---	---	---	---	---

- $start = 5$, $end = 4$ 이므로 반복문 탈출
- $start$ 가 $3 < arr[i]$ 를 만족하는 i 의 최솟값인 것을 확인 가능

이분 탐색

- 매개 변수 탐색
- 매개변수 탐색은 이분 탐색 아이디어를 한 단계 확장해서, 조건을 만족하는 최대값/최소 값을 찾는 데 사용한다.
 - 단순히 값의 위치를 찾는 것보다, 어떤 값이 조건을 만족하는지 확인하면서 답이 될 수 있는 후보 범위를 줄여 나간다.
 - 보통 middle값을 기준으로 조건을 만족 시킬 수 있는 `decision(int middle)` 함수를 따로 정의 하여 조건 만족 여부를 확인
- 복잡한 최적화 문제를 이분 탐색으로 바꿔 효율적으로 답을 찾을 수 있다.
- 예시
 - 이 문제를 풀 수 있는 최소 시간은?
 - `decision` 함수를 만족하는 최소의 middle 값 구하기
 - 이 조건을 만족하면서 버틸 수 있는 최대 무게는?
 - `decision` 함수를 만족하는 최대의 middle 값 구하기

이분 탐색

- 배열 `arr`의 크기가 `n`일 때,
- 반환값이 0인 경우
 - `key`가 배열의 가장 앞에 들어갈 때
 - lowerbound: `key <= arr[0]`
 - upperbound: `key < arr[0]`
- 반환값이 `n`인 경우:
 - `key`가 배열의 가장 뒤에 들어갈 때
 - lowerbound: `arr[n-1] < key`
 - upperbound: `arr[n-1] <= key`

이분 탐색

- 매개 변수 탐색 예시 문제 : <https://www.acmicpc.net/problem/2512>
- 우리가 원하는 것은 가능한 한 최대의 총 예산을 결정하는 일!
 - 즉 우리는 주어진 배열 (각 지방의 예산요청) 중 특정 값을 정하는 것이 아니라
 - 0~(지방의 예산요청의 최댓값)의 정수 중 하나를 골라야 한다!
- 단 조건이 예산을 정할 때 정해진 총액 이하여야 한다는 조건이 있다.
- 우리는 조건을 만족하는지 확인하면서 0~(지방의 예산요청의 최댓값)의 범위를 좁혀가면 된다!

이분 탐색

decision 함수

vector<int> price; // 지방의 예산요청이 정렬된 전역변수.

```
bool decision(int middle, int size, int budget) {
    int sum=0;
    for (int i = 0; i < size; i++)
        // 예산 요청액이 middle 보다 큰 경우
        // 일괄적으로 middle이 sum에 합 해짐
        sum += min(price[i], middle);
    // 조건에 만족하는지 안 하는지 반환
    return sum <= budget;
}
```

이분 탐색 (upperbound)

```
int search(int start, int end, int budget, int size) {
    while (start <= end) {
        int middle = start + (end - start) / 2;

        // 조건을 만족하면, 더 큰 쪽으로 범위를 줄여
        // 조건을 만족하는 middle를 한 번 더 찾아본다
        if (decision(middle, size, budget)) {
            start = middle + 1;
        } else {
            end = middle - 1;
        }
    }
    return end;
}
```

이분 탐색

- 일반 이분 탐색 : <https://www.acmicpc.net/problem/1920>
 - 예시 답변 : <http://boj.kr/da6299d0c9db4913942547eaf1b95017>
- 매개 변수 탐색 : <https://www.acmicpc.net/problem/2512>
 - 예시 답변 : <http://boj.kr/3f9bb4303ba640de89f14d41fe199d60>

복습 문제

- 이분탐색: <https://www.acmicpc.net/step/49>
- 분리집합: <https://www.acmicpc.net/step/14>